MINISTERUL EDUCAȚIEI NAȚIONALE

UNIVERSITATEA TEHNICĂ
DIN CLUJ-NAPOCA

FACULTATEA DE AUTOMATICĂ ȘI CALCULATOARE

GYM MANAGEMENT SYSTEM

-

INDUSTRIAL INFORMATICS PROJECT

Team: "GymTech"

Team leader:	Bogdan Dora	Group: 30332/1
Frontend developer:	Salajanu Mircea Vasile	Group: 30136/2
Backend developer:	Simionca Alexandru Petru Florin	Group: 30136/2
Full-Stack developer:	Kovacs Erik	Group: 30136/2
Tester:	Luidort Zsuzsa	Group: 30332/1

2026

MINISTERUL EDUCAȚIEI NAȚIONALE

UNIVERSITATEA TEHNICĂ
DIN CLUJ-NAPOCA

FACULTATEA DE AUTOMATICĂ ȘI CALCULATOARE

MINISTERUL EDUCAȚIEI NAȚIONALE

UNIVERSITATEA TEHNICĂ
DIN CLUJ-NAPOCA

FACULTATEA DE AUTOMATICĂ ȘI CALCULATOARE

PROJECT SUMMARY

of the project titled:

«Gym Management System»

1. Project requirements:

The project required the development of a **complex software application** following **Object-Oriented Programming (OOP) principles**, with a **structured software architecture**, a **graphical user interface (GUI)**, and **persistent data management**.

For this project, the team chose to develop a **Gym Management System**, designed to manage gym members, subscriptions, payments, attendance, and role-based user access through a functional web application.

2. Chosen solutions:

The project was implemented using a **three-tier software architecture**, separating the application into frontend, backend, and database layers.

The **frontend** was developed using **Angular**, providing a graphical user interface with role-based dashboards and interactive pages for users.

The **backend** was implemented using **ASP.NET Core Web API**, responsible for business logic, authentication, CRUD operations, and communication with the frontend through REST API endpoints.

For persistent data management, **SQL Server** and **Entity Framework Core** were used to store and manage application data. Additional tools such as **Swagger** were used for API testing, **Git/GitHub** for version control, and **Figma** for UI design.

3. Results obtained:

A functional version of the **Gym Management System** was successfully developed. The application includes a working login/sign-up system, role-

based authentication, operational dashboards for different user roles, CRUD functionalities for core entities, and database integration.

The system allows management of members, subscriptions, payments, attendance records, and role-based dashboard navigation, demonstrating the main functionalities required for gym management operations.

4. Testing and verification:

Testing and verification were performed using a combination of automated and manual testing methods. The backend was validated through **C# unit and integration tests**, while frontend components and route protection mechanisms were tested using **Vitest**. End-to-end workflows were verified using **Playwright** to simulate real user interactions across the application. Additionally, **Swagger** was used for API validation, and database operations, **authentication mechanisms, role-based access control**, and **CRUD functionalities** were tested to ensure correct system behavior and **reliable frontend-backend integration.**

MINISTERUL EDUCAȚIEI NAȚIONALE

UNIVERSITATEA TEHNICĂ
DIN CLUJ-NAPOCA

FACULTATEA DE AUTOMATICĂ ȘI CALCULATOARE

Table of Contents

1. Chapter 1 – INTRODUCTION

1.1 Problem Statement

Traditional gym management often relies on **manual processes** for handling members, subscriptions, payments, and attendance records. This can lead **to inefficiency, data inconsistency,** and difficulties in organizing gym operations.

The purpose of this project is to provide a **digital solution** that centralizes gym management activities into a single **web-based application** with **role-based access** and secure data management.

1.2 Objectives

Main objectives:

- Develop a functional gym management application;
- Implement role-based authentication and access control;
- Manage gym members and subscriptions;
- Track payments and attendance records;
- Provide dashboards for different user roles.

Secondary objectives:

- Create an intuitive graphical user interface;
- Ensure persistent data storage;
- Enable communication between frontend and backend;
- Verify and test application functionalities;
- Support scalable and maintainable application development.

1.3 Specifications

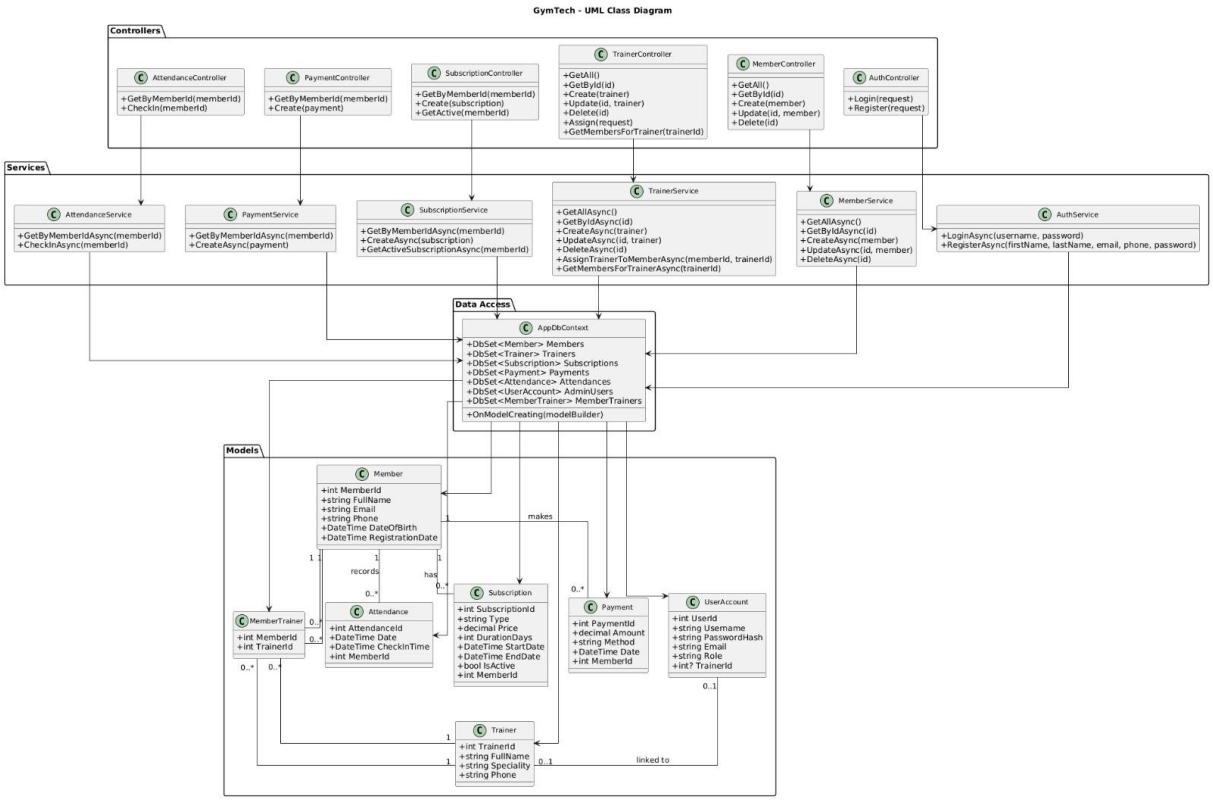
The application was developed using a structured software architecture and modern technologies.

- **Frontend:** Angular
- **Backend:** ASP.NET Core Web API
- **Database:** SQL Server
- **ORM:** Entity Framework Core
- **Programming languages:** TypeScript, C#
- **API testing:** Swagger
- **Frontend testing:** Vitest
- **End-to-End testing:** Playwright
- **Version control:** Git/GitHub
- **UI design:** Figma

The system implements **role-based authentication**, **CRUD operations**, **REST API communication**, **frontend-backend integration**, and **persistent relational database management,** ensuring secure and efficient gym management operations.

2. Chapter 2 – APPLICATION ARCHITECTURE

1. 2.1. Class Diagram – "What is the system made of?"



The UML Class Diagram illustrates the main classes of the Gym Management System and the relationships between them. The application is organized into four main layers: **Controllers**, **Services**, **Data Access**, and **Models**.

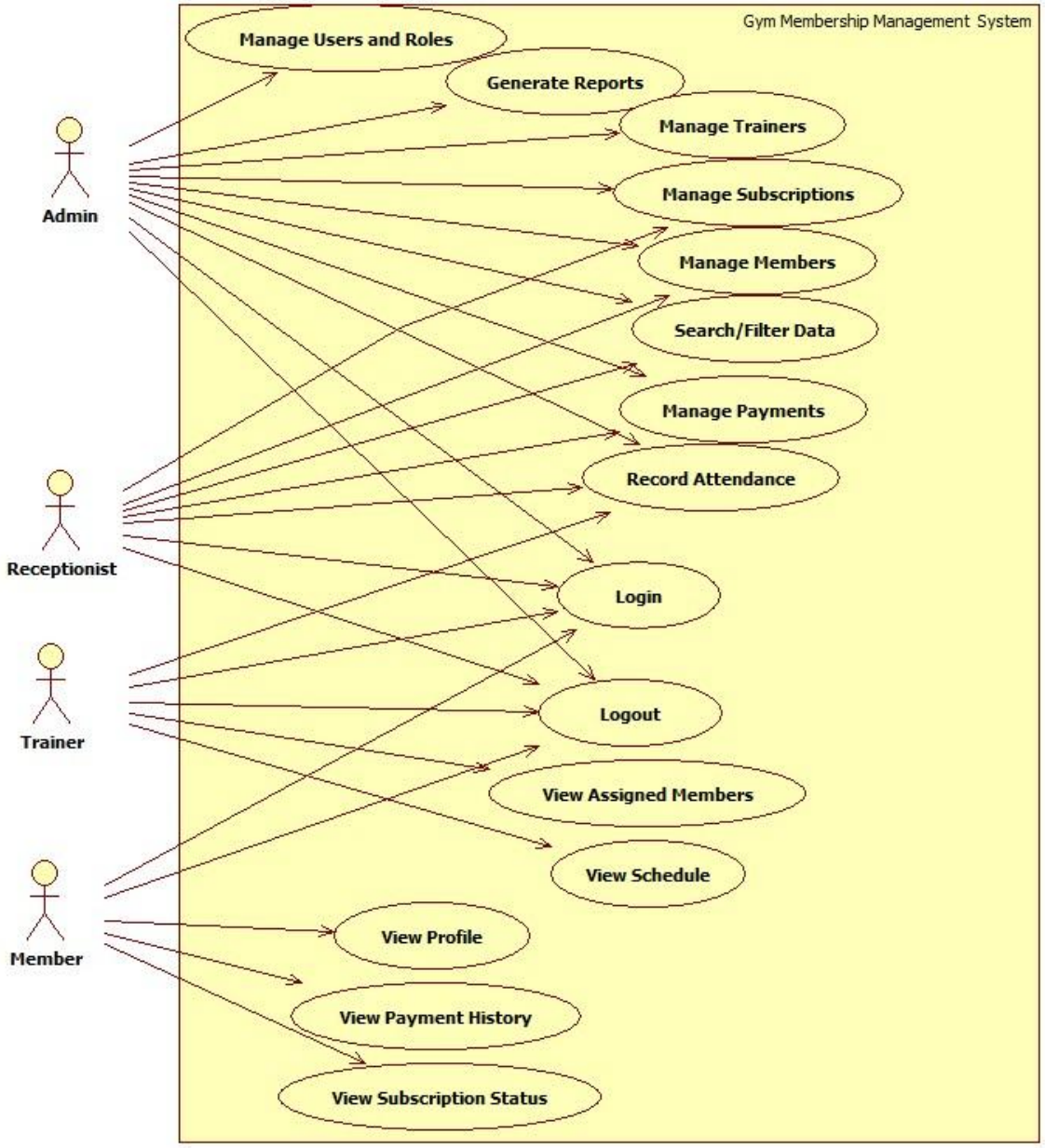
The **Controllers** handle incoming API requests and communicate with the corresponding service classes. The **Services** contain the business logic of the application, including member management, trainer assignment, subscription processing, payment handling, attendance tracking, and user authentication. The **AppDbContext** represents the data access layer and manages communication with the SQL Server database through Entity Framework Core.

The core entities of the system are **Member**, **Trainer**, **Subscription**, **Payment**, **Attendance**, **UserAccount**, and **MemberTrainer**. A member can have multiple subscriptions, payments, and attendance records, resulting in **one-to-many relationships**. The association between members and trainers is implemented through the **MemberTrainer** entity, creating a **many-to-many relationship**.

The **UserAccount** entity stores user credentials and role information required for authentication and authorization. This structure provides a clear separation of

responsibilities and supports the implementation of the application's main functionalities.

2. 2.2. Use Case Diagram – "What does the system do?"



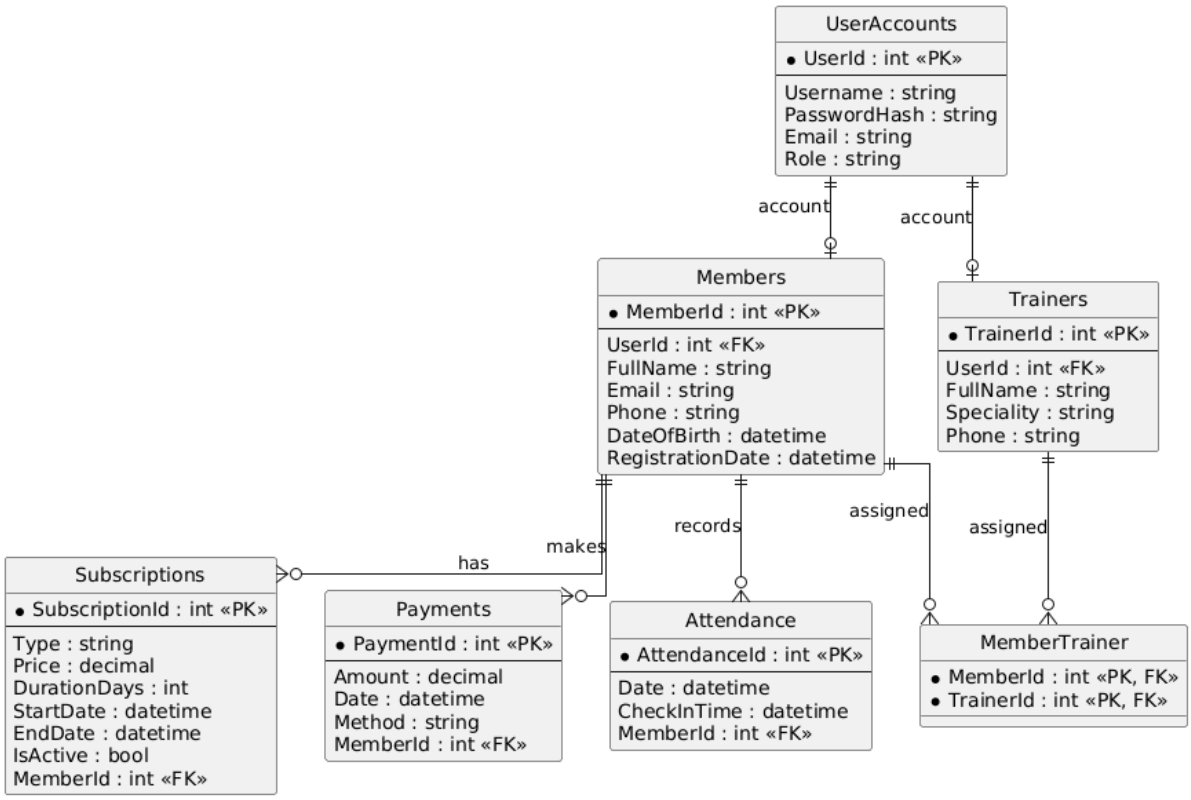
The **Use Case Diagram** illustrates the interactions between the system actors (Admin, Receptionist, Trainer, Member) and the available system functionalities. Each role has specific permissions and access rights within the Gym Management System.

- **Admin:** Manage members, trainers, subscriptions, payments, attendance, reports, and users/roles.

- **Receptionist:** Add and update members, view subscriptions, register payments, and check attendance.
- **Trainer:** View assigned members, monitor training activities, and access schedules.
- **Member:** View personal profile, subscription status, and payment history.

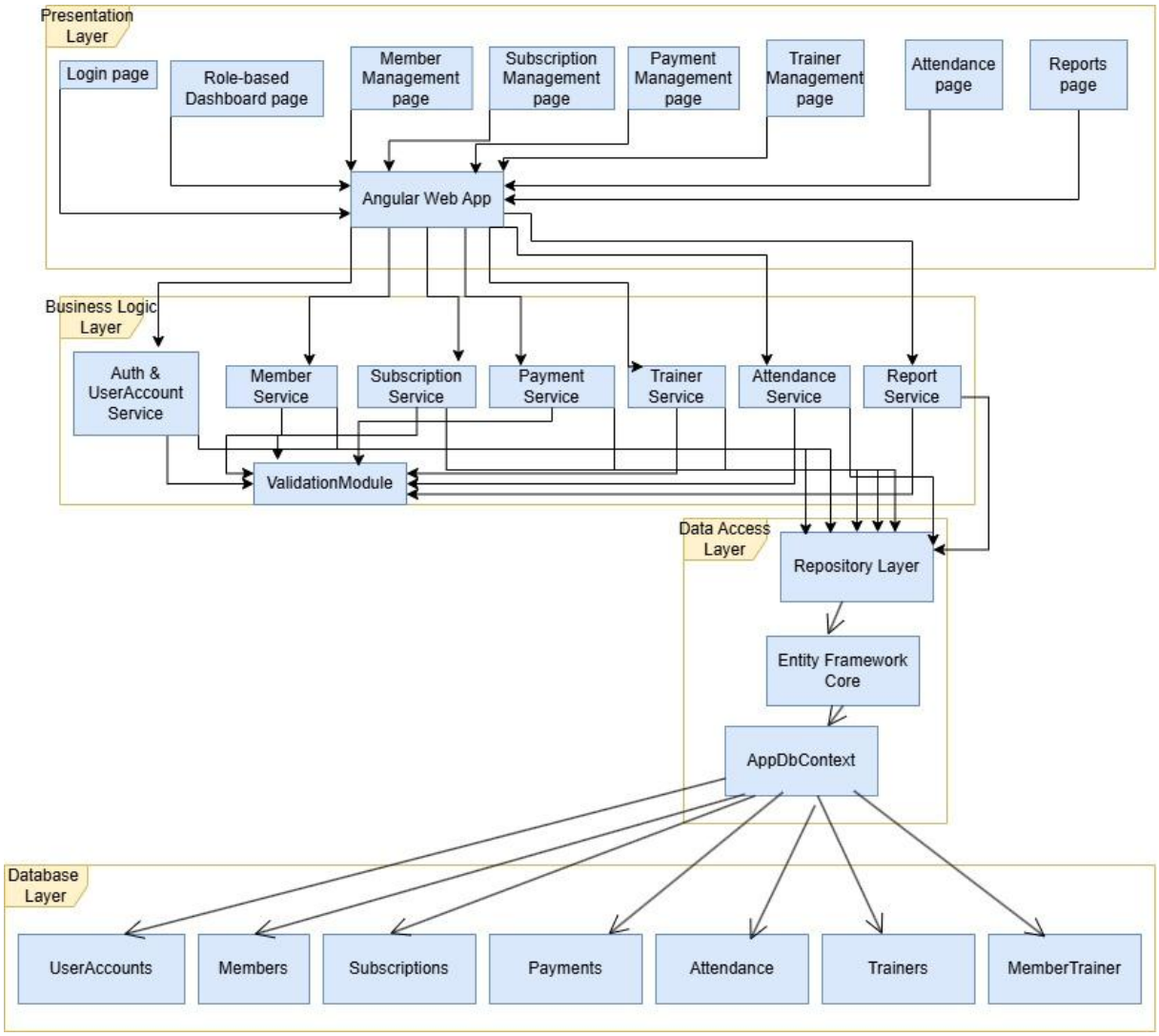
The Use Case Diagram demonstrates how different user roles interact with the system according to their responsibilities, ensuring controlled access to the application's functionalities.

3. 2.3. Database Diagram – "How is the data stored?"



The **ERD (Entity-Relationship Diagram)** illustrates the database structure of the Gym Management System, including the entities, primary keys, foreign keys, and relationships between tables. The **UserAccounts** table is responsible for authentication and role management, while the **Members** and **Trainers** tables store user-specific information. A member can have multiple subscriptions, payments, and attendance records, whereas the many-to-many relationship between members and trainers is managed through the **MemberTrainer** table.

4. 2.4. Architecture Diagram – "How is the system built?"



The **Architecture Diagram** illustrates the layered structure of the Gym Management System. The **Presentation Layer** is implemented using **Angular** and contains the user interface, including the login page, role-based dashboards, and management pages. The **Business Logic Layer** is developed using **ASP.NET Core Web API** and includes services responsible for authentication, member management, subscriptions, payments, trainers, attendance, and reporting. The **Data Access Layer** uses the **Repository Pattern**, **Entity Framework Core**, and **AppDbContext** to manage communication with the database. The **Database Layer** stores application data in **SQL Server** tables such as UserAccounts, Members, Subscriptions, Payments, Attendance, Trainers, and MemberTrainer. This layered architecture ensures **modularity**, **maintainability**, and a clear **separation of responsibilities**.

3. Chapter 3 – APPLICATION IMPLEMENTATION

3.1 Database Implementation

The database was implemented using **SQL Server and Entity Framework Core**. The main entities of the application are:

- Member
- Trainer
- Subscription
- Payment
- Attendance
- UserAccount
- MemberTrainer

Entity	Purpose
Member	Stores gym members and their personal details, such as full name, email, phone number, date of birth, and registration date.
Trainer	Stores trainers and their information, including full name, speciality, and phone number.
Subscription	Stores subscriptions created for members, including type, price, duration, start date, end date, active status, and member ID.
Payment	Stores payments made by members, including amount, payment method, date, and member ID.
Attendance	Stores gym attendance records, including date, check-in time, and member ID.
UserAccount	Stores authentication and role data, such as username, password, email, role, and optionally trainer ID.
MemberTrainer	Represents the many-to-many relationship between members and trainers.

Database Relationships

The main database relationships are:

- Member 1 -> many Subscriptions
- Member 1 -> many Payments
- Member 1 -> many Attendance records
- Member many -> many Trainer through MemberTrainer
- UserAccount 0..1 -> 0..1 Trainer

Entity Framework Core is used to configure and manage these relationships through the AppDbContext class.

```
public DbSet<Member> Members {get; set;}
public DbSet<Trainer> Trainers {get; set;}
public DbSet<Subscription> Subscriptions {get; set;}
public DbSet<Payment> Payments {get; set;}
public DbSet<Attendance> Attendances {get; set;}
public DbSet<UserAccount> AdminUsers {get; set;}
public DbSet<MemberTrainer> MemberTrainers {get; set;}
```

Entity Framework Core Setup

Entity Framework Core was configured in the backend project to connect the application to the SQL Server database. The connection string is defined in the application configuration file, and the database context is registered in Program.cs.

```
builder.Services.AddDbContext<AppDbContext>(options =>

options.UseSqlServer(builder.Configuration.GetConnectionString("DefaultConnection")));
```

The application uses migrations to keep the database schema synchronized with the C# models. Migrations are generated whenever the database structure changes and are applied to the SQL Server database.

```
Add-Migration MigrationName
Update-Database
```

This approach allows the database schema to evolve together with the backend models.

3.2 Backend Implementation

The backend of the GymTech application was implemented using **ASP.NET Core Web API**. The backend exposes REST API endpoints that are consumed by the Angular frontend. The application follows a layered structure, separating responsibilities between controllers, services, models, and database access.

```
Controllers -> Services -> AppDbContext -> SQL Server Database
```

Controllers are responsible for receiving HTTP requests from the frontend, while services contain the business logic. The AppDbContext class is used by Entity Framework Core to communicate with the SQL Server database.

Backend Services and Business Logic

The **business logic** of the application is implemented inside service classes. Each main entity has a corresponding service that handles its operations.

- AuthService
- MemberService
- TrainerService
- SubscriptionService
- PaymentService
- AttendanceService

Service	Responsibility
AuthService	Handles login and registration logic.
MemberService	Handles operations related to gym members, such as retrieving, creating, updating, and deleting members.
TrainerService	Handles trainer management and the assignment of trainers to members.
SubscriptionService	Manages member subscriptions, including creating new subscriptions and retrieving active subscriptions.
PaymentService	Handles payment records for members.
AttendanceService	Manages attendance history and member check-ins.

This separation keeps controllers simple and moves the actual business logic into reusable service classes.

API Endpoints and CRUD Operations

The backend exposes REST API endpoints through controllers. These endpoints allow the frontend to perform CRUD operations.

```
GET members
GET member by ID
POST create member
PUT update member
DELETE member

GET trainers
POST create trainer
PUT update trainer
DELETE trainer
```

```
POST login
POST register

GET subscriptions by member ID
POST create subscription

GET payments by member ID
POST record payment

GET attendance by member ID
POST check-in
```

The controllers receive requests from the frontend and call the corresponding service methods.

```
[ApiController]
[Route("api/[controller]")]
public class MemberController : ControllerBase
{
    private readonly MemberService _memberService;

    public MemberController(MemberService memberService)
    {
        _memberService = memberService;
    }

    [HttpGet]
    public async Task<IActionResult> GetAll()
    {
        var members = await _memberService.GetAllAsync();
        return Ok(members);
    }
}
```

Authentication Logic

Authentication is implemented in the backend using the UserAccount entity. Users are stored in the AdminUsers table and have a role assigned to them.

Available roles include:

- Admin
- Receptionist
- Trainer
- Member

During login, the backend checks if the username or email and password match an existing user account. If the credentials are valid, the backend returns the user role to the frontend. The frontend then redirects the user to the correct dashboard based on the role.

Role	Redirected Dashboard
Admin	Admin Dashboard
Receptionist	Receptionist Dashboard
Trainer	Trainer Dashboard
Member	Member Dashboard

During registration, the member account is created automatically. The username is generated from the email address by taking the part before @. This avoids adding an extra username field in the signup form while still keeping the Username column in the database.

```
email: alex.popescu@gmail.com
generated username: alex.popescu
```

Swagger Testing

Swagger was enabled in the backend project to test API endpoints directly from the browser. It was useful for testing login, registration, member management, trainer management, subscriptions, payments, and attendance endpoints.

Swagger allows the developer to send test requests to the API and inspect the returned responses without using the frontend.

```
POST /api/Auth/login

{
  "username": "admin",
  "password": "admin123"
}
```

Swagger helped verify that the backend was working correctly before connecting it to the Angular frontend.

3.3 Frontend Implementation

3.3.1 Angular Pages Implemented

The GymTech frontend is a role-based single-page application (SPA) built with Angular 18 (standalone components).

It implements:

-Login/Signup Page - Unified authentication with client-side validation
-4 Role-Specific Dashboards:
Admin Dashboard (comprehensive system management)
Member Dashboard (personal account management)
Trainer Dashboard (client management)
Receptionist Dashboard (reception operations)

3.3.2 Dashboards and Role-Based UI

***Admin Dashboard**:
-Home: Statistics cards displaying Total Members, Active Trainers, Active Subscriptions, Total Revenue
-Sidebar Navigation with management features:
Manage Members, Trainers, Subscriptions, Payments, Users & Roles
Record Attendance
Search & Filter functionality
Generate Reports (with income/attendance comparisons)

***Member Dashboard**
Home: Current subscription status and days remaining
Account Pages: Profile view, Subscription status, Payment history, Attendance history
Subscription Management: Browse and subscribe to available plans (Standard/Premium/Trainer)
Trainer Selection: For trainer plans, members select their preferred trainer

***Trainer Dashboard**
View assigned members
Search members functionality
Record attendance for members
View training schedule

***Receptionist Dashboard**
Member search and editing
Member check-in (attendance recording)
Subscribe members to plans
Process payments
Advanced search with filters

Role-Based Routing:
/login → Authentication
/dashboard → Admin (protected)
/member-dashboard → Member (protected)
/trainer-dashboard → Trainer (protected)
/receptionist-dashboard → Receptionist (protected)

3.3.3 Forms, Navigation, and Routing

Form Architecture
*Two-way binding using Angular's FormsModule
*Client-side validation (required fields, password matching)

*Error/success message display with real-time feedback
*Loading states on form submission

Navigation Structure
*Sidebar-based feature switching within each dashboard
*Single-page rendering: All features render in the same component
*AuthGuard protection: Routes check authentication status via localStorage
*Automatic role-based redirect: Login routes users to their dashboard

Routing Protection

"export const authGuard: CanActivateFn = () => {
const authService = inject(AuthService);
if (authService.isLoggedIn()) return true;
router.navigate(['/login']);
return false;
};"

Session Persistence: User data stored in localStorage under key gymtech_user with role, username, userId, memberId, and trainerId.

3.3.4 Frontend Design Improvements

*Dark/Light Mode Theme System
*ThemeService: Signal-based state management for reactive theme switching
*Persistent Theme: localStorage saves user preference (default: dark mode)
*Toggle Button: Fixed bottom-left corner for easy access
*CSS Class Management: [light-theme] class applied to body for stylesheet switching

Implementation:

"constructor() {
const savedTheme = localStorage.getItem('theme') ?? 'dark';
this.themeSignal.set(savedTheme);
document.body.classList.toggle('light-theme', savedTheme === 'light');
}"

Design System
*Primary Accent: #E85B8A (Pink/Magenta) - used for active states, highlights, and CTAs
*Dark Colors: #1a1a1a (main), #0d0d0d (sidebar), #000000 (navbar)
*Visual Effects: Hover states with color transitions, elevation (transform), and box shadows
*Responsive Grid: Used for stat cards and report cards

UI Components
*Stat Cards: Icons + labels + values in grid layout

*Buttons: Primary style with hover effects and loading states
*Forms: Labels, inputs, error messages, validation feedback
*Report Cards: Comparison metrics with percentage changes and directional indicators

3.3.5 Interaction with Backend API from Frontend

Service Layer Architecture
Each resource has a dedicated, injectable service using **Angular's HttpClient:**

Core Services:
[AuthService] - Login, registration, session management
[MemberService]- Member CRUD, search
[TrainerService] - Trainer management, assignments
[SubscriptionService] - Plan retrieval, subscription operations
[PaymentService] - Payment tracking
[AttendanceService] - Attendance recording
[StatsService] - Dashboard statistics
[UserService]- User/role management

API Base URL: http://localhost:5062/api/

*Auth**:** POST /auth/login, POST /auth/register
*Member: GET /member, GET /member/{id}, POST /member, PUT /member/{id}, DELETE /member/{id}, GET /member/search?query
*Subscription: GET /subscription/plans, POST /subscription, GET /subscription/member/{memberId}
*Payment: GET /payment/all, GET /payment/member/{memberId}, POST /payment
*Trainer: GET /trainer, POST /trainer, POST /trainer/assign
*Attendance: GET /attendance/member/{memberId}, POST /attendance
*Stats: GET /stats

Data Flow Pattern

Example: **Member Subscription**
User selects subscription plan on Member Dashboard
Form validates inputs (memberId, plan details)
SubscriptionServiceCreate() sends **POST/api/subscription**
Success: Displays confirmation, reloads subscriptions
Error: Shows backend error message to user

Error Handling

```
"this.service.operation().subscribe({
next: (response) => { /* handle success */ },
error: (err) => {
this.errorMessage = err?.error?.message ?? 'Operation failed';
}
});"
```

Session Management

// After login
"localStorage.setItem('gymtech_user', JSON.stringify({
username, role, userId, memberId, trainerId
}));"

// On logout
"localStorage.removeItem('gymtech_user');"

3.3.6 Frontend Technology Stack

Framework: Angular 18 (standalone components)
HTTP Client: Angular HttpClient with RxJS Observables
State Management: Angular signals (reactive primitives)
Routing: Angular Router with route guards
Styling: CSS3 with theme variables and dark mode support
Forms: FormsModule with two-way binding
Build: Angular CLI

3.4 Full-Stack Integration and Role-Based Authentication

The application was implemented as a full-stack web system, using **Angular** for the frontend, **ASP.NET Core Web API** for the backend, and **SQL Server** as the database. The main implementation focus was the connection between the user interface, the backend API, and the database, especially for authentication, role detection, and dashboard redirection.

3.4.1 Frontend-Backend Synchronization

The frontend and backend communicate through REST API endpoints. The Angular application sends HTTP requests to the ASP.NET Core Web API, while the backend processes the requests, accesses the database through Entity Framework Core, and returns the response in JSON format.

In Angular, services are used to centralize API calls. This keeps the components cleaner and makes the communication between the frontend and backend easier to manage.

Example of a frontend service method:

```
login(loginData: any) {
  return this.http.post<any>('https://localhost:xxxx/api/Auth/login', loginData);
}
```

The same approach is used for other application features, such as retrieving members, trainers, subscriptions, payments, and attendance records. After adding, updating, or deleting data, the frontend refreshes the displayed information so that the interface remains synchronized with the database.

Example of loading data from the backend:

```
loadMembers() {
  this.memberService.getMembers().subscribe({
    next: (data) => {
      this.members = data;
    },
    error: (err) => {
      console.error('Error loading members:', err);
    }
  });
}
```

This ensures that the data displayed in the Angular interface corresponds to the latest information stored in the SQL Server database.

3.4.2 Login and Register Implementation

The login and register functionalities were implemented to allow users to access the system using personal accounts.

For registration, the user completes a form in the Angular frontend. The data is sent to the backend, where a new user account is created and stored in the database.

Example frontend register call:

```
register(registerData: any) {
  return this.http.post<any>('https://localhost:xxxx/api/Auth/register', registerData);
}
```

On the backend side, the received data is saved using Entity Framework Core.

Example backend register logic:

```
[HttpPost("register")]
public IActionResult Register(RegisterDto registerDto)
{
    var user = new UserAccount
    {
        Username = registerDto.Username,
        Email = registerDto.Email,
        Password = registerDto.Password,
        Role = registerDto.Role
    };

    _context.UserAccounts.Add(user);
    _context.SaveChanges();
```

```
    return Ok(new { message = "User registered successfully" });
}
```

For login, the user enters the email and password in the frontend form. These credentials are sent to the backend, where they are checked against the existing accounts from the database.

Example backend login logic:

```
[HttpPost("login")]
public IActionResult Login(LoginDto loginDto)
{
    var user = _context.UserAccounts
        .FirstOrDefault(u => u.Email == loginDto.Email
                        && u.Password == loginDto.Password);

    if (user == null)
    {
        return Unauthorized(new { message = "Invalid email or password" });
    }

    return Ok(new
    {
        id = user.Id,
        username = user.Username,
        email = user.Email,
        role = user.Role
    });
}
```

If the credentials are valid, the backend returns the user information together with the user role. This response is then used by the frontend to redirect the user to the correct dashboard.

3.4.3 Role-Based Authentication

The application uses role-based authentication in order to separate the access level of each user type. The main roles used in the system are:

- **Admin**
- **Trainer**
- **Receptionist**
- **Member**

Each role has access to a different dashboard and different functionalities. This makes the application closer to a real gym management system, where each user type has different responsibilities.

For example, the Admin can manage the main system data, the Trainer can view assigned members and schedules, the Receptionist can manage members and subscriptions, while the Member can view personal information.

The role is stored in the database as part of the user account.

Example user account model:

```
public class UserAccount
{
    public int Id { get; set; }

    public string Username { get; set; }

    public string Email { get; set; }

    public string Password { get; set; }

    public string Role { get; set; }
}
```

3.4.4 Role Detection from Database

The user role is detected during the login process. When the user logs in, the backend searches for the account in the database. If the account exists, the role stored in the database is returned to the frontend.

Example response returned by the backend:

```
{
  "id": 1,
  "username": "admin",
  "email": "admin@gymtech.com",
  "role": "Admin"
}
```

This role is important because the frontend uses it to decide what dashboard should be displayed. In this way, the application does not redirect all users to the same page, but adapts the interface according to the authenticated user.

On the frontend side, the received user data can be stored locally:

```
this.authService.login(this.loginData).subscribe({
  next: (user) => {
    localStorage.setItem('user', JSON.stringify(user));
    this.redirectUser(user.role);
  },
  error: () => {
    this.errorMessage = 'Invalid email or password';
  }
});
```

3.4.5 Dashboard Redirection Based on User Role

After a successful login, the user is automatically redirected to the correct dashboard based on their role.

Example Angular redirection logic:

```
redirectUser(role: string) {
  if (role === 'Admin') {
    this.router.navigate(['/admin-dashboard']);
  }
  else if (role === 'Trainer') {
    this.router.navigate(['/trainer-dashboard']);
  }
  else if (role === 'Receptionist') {
    this.router.navigate(['/receptionist-dashboard']);
  }
  else if (role === 'Member') {
    this.router.navigate(['/member-dashboard']);
  }
  else {
    this.router.navigate(['/login']);
  }
}
```

This improves the user experience because every user is taken directly to the page that matches their responsibilities inside the system.

The routing is handled in Angular, where each dashboard has its own route.

Example route configuration:

```
const routes: Routes = [
  { path: 'admin-dashboard', component: AdminDashboardComponent },
  { path: 'trainer-dashboard', component: TrainerDashboardComponent },
  { path: 'receptionist-dashboard', component: ReceptionistDashboardComponent },
  { path: 'member-dashboard', component: MemberDashboardComponent },
  { path: 'login', component: LoginComponent }
];
```

3.4.6 Integration Issues and Solutions

During the integration process, several issues appeared between the frontend and backend.

One issue was related to the difference between the data structure expected by the frontend and the data returned by the backend. For example, some field names or object structures did not match at first. This was solved by checking the API responses in Swagger and updating the Angular models and services according to the backend response.

Another issue was related to the API routes. Some frontend service methods did not call the correct backend endpoint. This was solved by testing each endpoint in Swagger and then updating the URLs used in the Angular services.

Example API URL used inside an Angular service:

```
private apiUrl = 'https://localhost:xxxx/api/Member';
```

There were also synchronization issues where changes made in the backend or database were not immediately visible in the frontend. This was solved by reloading the data after each operation.

Example of refreshing the list after deleting a member:

```
deleteMember(id: number) {
  this.memberService.deleteMember(id).subscribe({
    next: () => {
      this.loadMembers();
    },
    error: (err) => {
      console.error('Error deleting member:', err);
    }
  });
}
```

Another common issue was related to running both projects at the same time. The backend had to be started first, so that the API was available, and then the Angular frontend could be started. The connection was verified by testing the backend endpoints in Swagger and checking if the frontend correctly displayed the returned data.

3.4.7 Summary

Overall, the full-stack integration successfully connected the **Angular frontend**, **ASP.NET Core Web API backend**, and **SQL Server database** into a unified system. **User authentication, role detection, dashboard redirection**, and **CRUD operations** were implemented across all application layers. The final result is a functional gym management application where users are authenticated, their roles are retrieved from the database, and they are automatically redirected to the dashboard corresponding to their responsibilities.

4. Chapter 4 – APPLICATION TESTING

4.1 Testing Strategy and Methodology

The validation phase of the GymTech application was executed using a structured, multi-tier testing strategy designed to verify the reliability of both the frontend presentation layer and the backend service architecture. Quality assurance was treated as a shared, collaborative effort between the developers and the tester, with a strong focus on automated code verification to maintain project stability over time.

The testing strategy was divided across the primary programming languages driving the application stack:

- **C# Testing Architecture (Backend Layer):** White-box testing was applied to the server-side environment. Code-based unit tests and integration tests were implemented to directly isolate the business logic within the service classes (such as `AuthService` and `MemberService`) and to ensure that database queries executed smoothly against the `AppDbContext` without violating relational rules.
- **TypeScript Testing Architecture (Frontend Layer):** Isolated unit tests were written to target client-side logical structures. Functional route guards (`authGuard`) and state tracking systems driven by Angular Signals were tested using Vitest to ensure code stability inside the browser.
- **Full-Stack Integration (End-to-End Testing):** To test the interaction between both languages over the live connection network, automated end-to-end (E2E) workflows were constructed using Playwright. These scripts drove a headless browser engine to interact with the UI, mimicking real user paths across forms, data tables, and authentication views.

Complementing the automated code layers, **Swagger UI** was utilized to manually execute isolated HTTP requests against the REST API endpoints during early deployment phases, ensuring response schemas matched expectations before frontend connection began.

4.2 Application Workflow Overview

In practice, the application functions by hosting a C# ASP.NET Core REST API that handles business logic and persistent SQL Server transactions, while a TypeScript Angular frontend acts as the graphical interface. When users interact with the client dashboards, secure HTTP requests carrying JWT tokens are continuously transmitted back and forth across the network layer to manipulate database records seamlessly in real time.

4.3 Test Cases

Authentication and Access Control Gates

- **Role-Based Dashboard Redirection:** Code scripts tested the execution of the `POST /api/Auth/login` endpoint. The test case verified that when legitimate credentials are provided, the backend successfully cross-references the account table, returns the matching user role, and the frontend accurately handles the routing logic to mount the correct dashboard grid layout (Admin, Receptionist, Trainer, or Member Dashboard).
- **Route Security Isolation:** Functional unit tests targeted the `authGuard` using Vitest mocks. The test cases checked that unauthorized users attempting to skip login pages were actively intercepted at the routing engine level and safely redirected back to the login screen.

Service CRUD Operations and Relational Data Validation

- **Member Lifecycle Operations:** Test scenarios validated the execution loops inside the `MemberController` and `MemberService`. The code automatically filled mandatory fields, triggered a postback, and verified that a new database row containing the member's personal data was committed cleanly to SQL Server and accurately displayed in the frontend data view.
- **One-to-Many Contract Linkages:** Testing scripts validated the workflows that connect distinct entities inside the domain model. Test cases successfully verified that adding a subscription package (defining Type, Price, and Duration) programmatically associated the entry with the correct parent `MemberId` foreign key, satisfying Entity Framework Core validation properties.

4.4 Issues Encountered and Solutions

Because the core architecture of the application was structurally sound from the initial compilation phases, the testing process encountered very few exceptions or functional application bugs. The original core codebase remained completely intact and did not require deep structural rewrites or intrusive modifications.

Instead, the QA lifecycle focused on refining the user experience, clarifying workflows, and ensuring overall application clarity. The minor anomalies identified during testing were not fundamental system issues, but rather opportunities to make usage clearer and easier for the end-user.

Rather than treating defect resolution as an isolated logging process, these improvements were handled through direct, human-to-human team communication. The tester discussed these observations directly with the backend and frontend developers, and through this collaborative feedback loop, minor visual boundaries, form handling rules, and test script configurations were refined to achieve a highly polished and intuitive end product.

4.5 Final Testing Verification Matrix

The combined testing pipeline concluded with a 100% success rate across all frontend and backend code files, confirming complete feature validation:

Scope	Target Component / File	Verification Method	Status
Data Integrity	Relational Database Entities & Constraints	C# In-Memory SQLite Constraint Testing	**PASSED**
API Architecture	Controller Endpoints & Route Mapping	C# `WebApplicationFactory` Host Testing	**PASSED**
Client Routing	Functional `authGuard.ts` Logic	TypeScript Vitest Mock Testing	**PASSED**
UI State Systems	Global Visual `theme.service.ts` Signals	TypeScript Vitest Reactive Assertions	**PASSED**
Network Gatekeeper	Security Header `auth.interceptor.ts` Injection	TypeScript Vitest Mock HTTP Pipe Testing	**PASSED**
User Workflows	Complete Full-Stack Integration Lifecycles	Playwright Headless Browser Automation	**PASSED**

To provide a clear architectural overview of this matrix, each testing scope is defined below:

- **Data Architecture Verification (Data Integrity):** Relational database rules are verified using an in-memory database configuration to ensure primary keys, foreign keys, and cascading relationships hold true during data creation operations.

- **In-Memory Hosting (API Architecture):** Utilizes `WebApplicationFactory` to host the C# server inside memory tracks during validation cycles, allowing automated scripts to transmit functional HTTP requests directly to your API controllers and inspect responses instantly.
- **Navigation Enforcements (Client Routing):** Isolates and runs the application's `authGuard.ts` layer to ensure page views stay securely locked down and unauthenticated users are safely deflected.
- **Application States (UI State Systems):** Ensures your Angular Signals react correctly inside `theme.service.ts` to instantly transform visual layouts and sync preferences with local browser storage.
- **Credential Distribution (Network Gatekeeper):** Tests the functional `auth.interceptor.ts` stream layout to confirm that it automatically catches outgoing application requests and attaches the bearer JWT token seamlessly.
- **Full Integration Workflow (User Workflows):** Driven by your automated Playwright suites, this ensures that the complete system functions as one cohesive engine from user login down to administrative management listings.

5. CONCLUSIONS

The GymTech project successfully achieved its main objective of developing a **functional gym management web application**. The system integrates an **Angular frontend**, an **ASP.NET Core Web API backend**, and a **SQL Server database.** The application supports **role-based authentication**, **dashboard redirection**, **CRUD operations**, member and subscription management, payments, attendance tracking, and frontend-backend communication through **REST API endpoints**. Testing was performed through backend tests, frontend tests, end-to-end workflows, Swagger validation, and database verification. Overall, the project demonstrates a **structured, maintainable, and functional full-stack software solution** for gym management.

6. Appendix 1 – Source Code

Source Code Repository: *The complete source code of the project is available in the GitHub repository used for version control, collaboration, and project management throughout the development process.*

- **GitHub Repository:** `https://github.com/Dora-14/GymTech`
- **Branch:** `main`

The repository contains both the **Angular frontend application** and the **ASP.NET Core Web API backend**, including database models, Entity Framework Core migrations, authentication modules, testing files, and project documentation.